

Addendum: reproducción del Game Day de Chaos Engineering en cluster kubernetes real

MASOBAP2026 · Observabilidad U2 · Fork `jorecof/chaos_k8s` · versión 2 — incluye el Experimento 2 (HTTPChaos)

Este addendum documenta la reproducción completa del laboratorio de Chaos Engineering original (diseñado para GKE) sobre un cluster kubernetes propio de 3 nodos, con datos y evidencia reales capturados durante la ejecución (no simulados). Incluye: arquitectura real desplegada, verificación de estado estable antes de inyectar fallas, resultados reales del Experimento 1 (latencia de red) con el análisis de trazas Jaeger que explica el efecto a nivel de spans, y —nuevo en esta versión— el Experimento 2 (error rate por HTTPChaos), cuyos dos hallazgos principales no estaban previstos en el diseño del experimento y son el aporte más valioso de la reproducción.

1. Arquitectura real desplegada

A diferencia del docker-compose del enunciado original, el laboratorio corre sobre un cluster kubernetes de 3 VMs (VirtualBox, Debian 12 arm64, containerd 2.3.3, Calico v3.32.1, K8s v1.35). Las cargas con estado (registry, Postgres, Prometheus, Grafana, Jaeger) quedan ancladas a `kube-cp` vía `nodeSelector: rol=observabilidad`; el otel-collector corre como DaemonSet (una réplica por nodo) para que el scrape de métricas por pod nunca dependa de a cuál réplica enruta el Service.

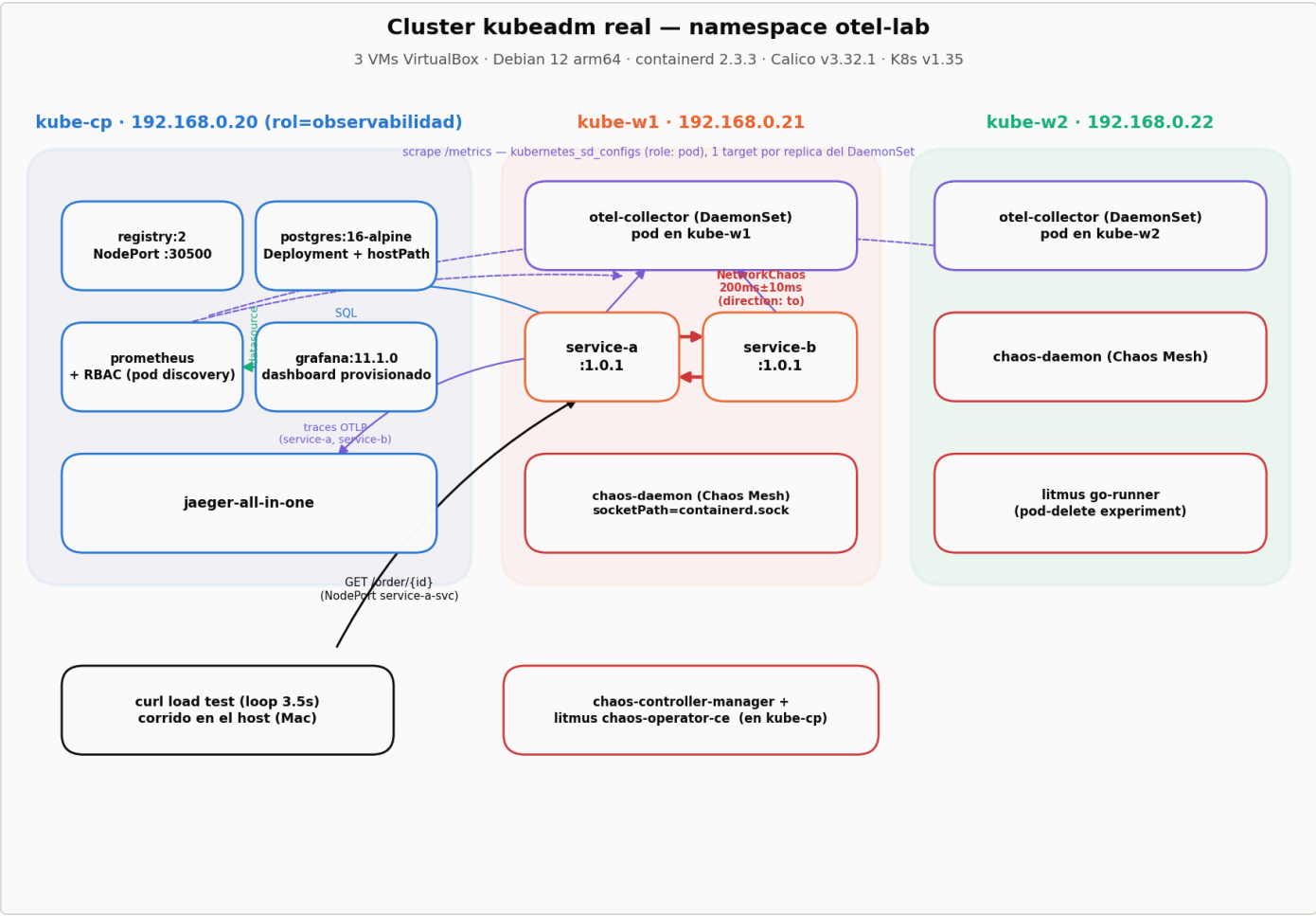


Figura 1. Topología real del cluster y flujo de datos entre componentes.

2. Verificación de estado estable (antes del chaos)

Antes de inyectar fallas se revisaron Prometheus, Jaeger y las apps para confirmar comportamiento normal. Esa revisión encontró y corrigió, sobre el sistema real, tres problemas que hubieran invalidado cualquier medición posterior:

PROBLEMA	CAUSA RAÍZ	CORRECCIÓN
Propagación de trace context rota entre service-a y service-b	<code>opentelemetry-instrumentation-fastapi >=0.48b0</code> ya no engancha con el patch global <code>instrument()</code> llamado antes de crear la app	<code>FastAPIInstrumentor.instrument_app(app, ...)</code> llamado sobre la instancia, después de crearla
Métricas de service-b ausentes o intermitentes en Prometheus	Prometheus scrapeaba el Service del otel-collector (round-robin sobre 3 réplicas DaemonSet) en vez de cada pod	<code>kubernetes_sd_configs (role: pod)</code> + RBAC nuevo (ServiceAccount + ClusterRole) para scrape por pod
Duración de traza sin sentido en Jaeger (57m 60s)	Desfasaje de reloj entre las 3 VMs (NTP desactivado)	<code>timedatectl set-ntp true</code> + restart de <code>systemd-timesyncd</code> en los 3 nodos

Con estas correcciones aplicadas y verificadas (traza correlacionada end-to-end en Jaeger, métricas de las 3 apps visibles en Prometheus, relojes sincronizados), se capturó la línea base y se procedió con los experimentos de chaos.

3. Experimento 1 — NetworkChaos (200 ms ±10 ms sobre service-b)

Chaos Mesh NetworkChaos, acción `delay`, `direction: to` (afecta tráfico entrante y saliente de los pods de service-b), `duration: 5m` con rollback automático por TTL. Carga: `curl` en loop cada ~3,5 s contra `GET /order/{order_id}` (service-a → service-b → Postgres), corrido en el host real durante la ventana del experimento.

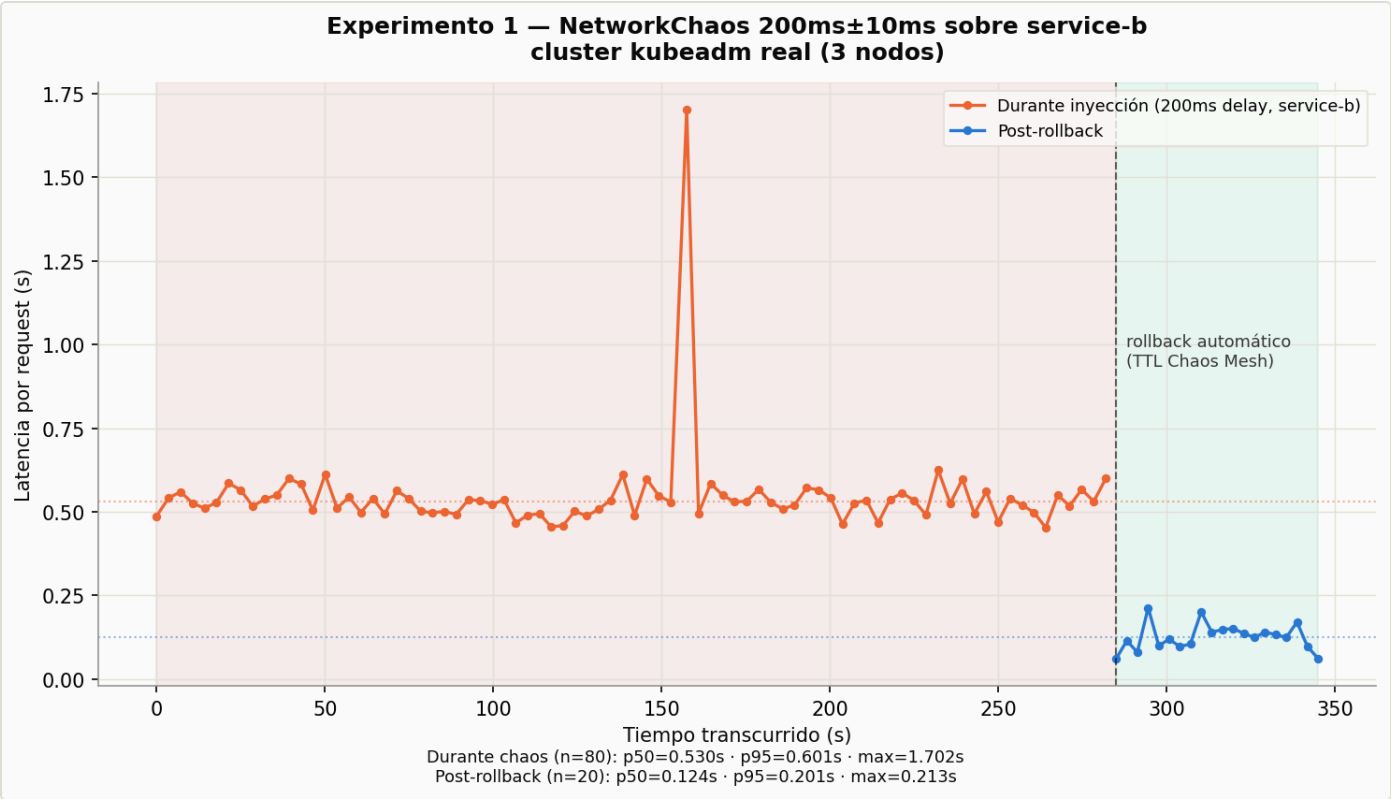


Figura 2. Latencia por request, datos reales del cliente curl (n = 100).

FASE	N	P50	P95	MÁX
Durante inyección (chaos activo)	80	0,530 s	0,601 s	1,702 s
Post-rollback (chaos removido)	20	0,124 s	0,201 s	0,213 s

El rollback automático por TTL de Chaos Mesh se confirma empíricamente: p50 baja de 0,530 s a 0,124 s (~4,3×) apenas expira la ventana de 5 minutos, sin intervención manual. El pico aislado de 1,702 s durante la inyección es una cola esperable (jitter +10 ms más contención puntual), no un patrón sostenido. **Este comportamiento limpio es el punto de comparación para el Experimento 2**, donde el mismo mecanismo de rollback falla.

4. Análisis de trazas Jaeger: efecto en los spans

Se compararon dos trazas reales del mismo endpoint (`GET /order/{order_id}`, servicio service-a), una capturada durante la inyección de NetworkChaos y otra después del rollback, para ubicar exactamente dónde vive la latencia agregada.

TRAZA	TRACE ID	INICIO	DURACIÓN	CONDICIÓN
Normal	12ca4308...f209b90	20:04:29.743	58,99 ms	Post-rollback
Con chaos	78d7f724...b63ddab7	20:03:26.231	579,89 ms	NetworkChaos activo

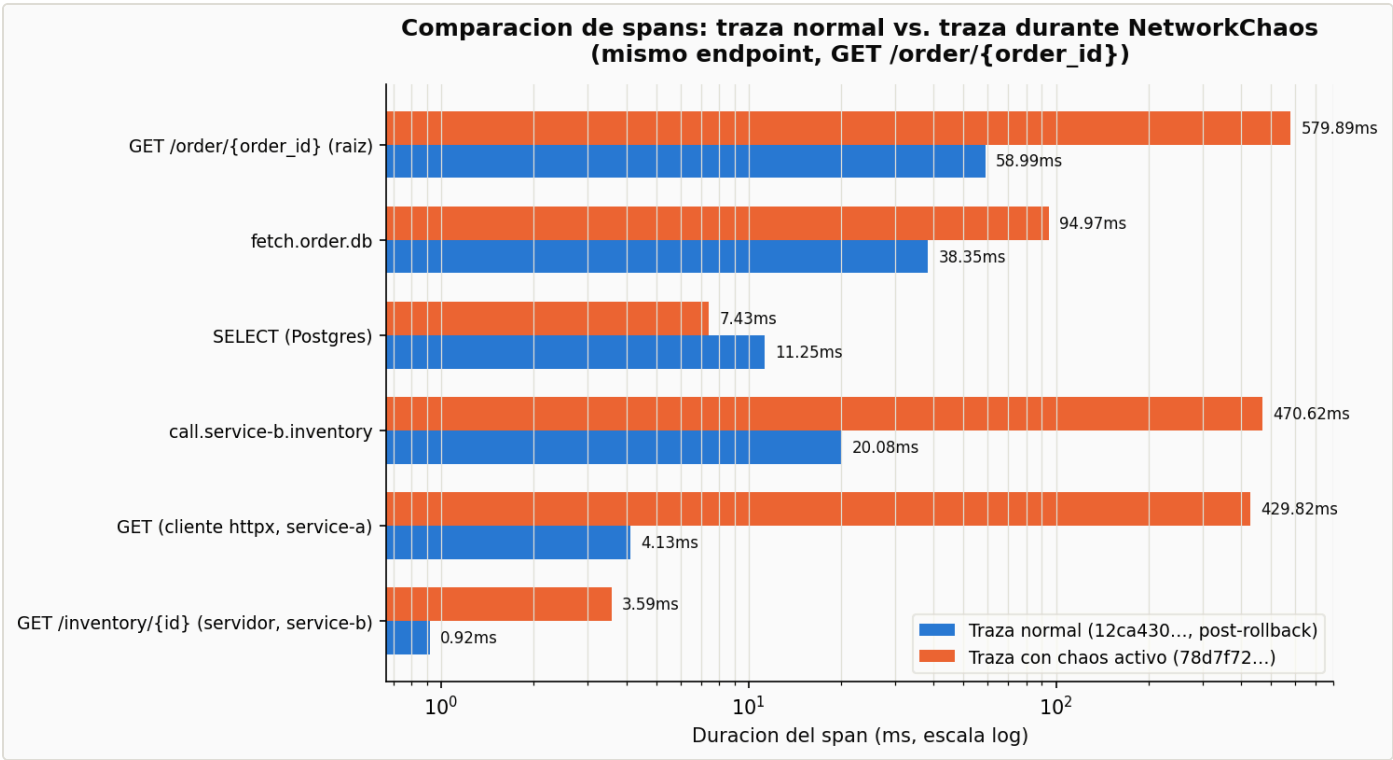


Figura 3. Duración por span, misma operación, traza normal vs. traza con chaos activo.

El desglose por span muestra que el 96 % del tiempo agregado por el chaos se concentra en un único salto: el span cliente `GET` (httpx, dentro de service-a, llamando a service-b) pasa de 4,13 ms a 429,82 ms (+425,7 ms), mientras que el span servidor correspondiente en service-b (`GET /inventory/{product_id}`) apenas crece de 0,92 ms a 3,59 ms. La diferencia entre el tiempo visto por el cliente y el tiempo de procesamiento real del servidor ($429,82 - 3,59 = 426,2$ ms) es, casi milimétricamente, el efecto de la falla inyectada: NetworkChaos con `direction: to` aplica el delay de $200\text{ ms} \pm 10\text{ ms}$ en ambas direcciones del tráfico de service-b (ida y vuelta), por lo que el round-trip crece en aproximadamente $2 \times 200\text{ ms} = 400\text{ ms}$ más jitter — consistente con los 426 ms observados. El resto de la traza (SELECT a Postgres, cache hit en service-b, spans internos de envío HTTP) no se mueve de forma significativa, lo que confirma que el blast radius del experimento quedó contenido al salto service-a $\rightarrow$ service-b, tal como definía el NetworkChaos.

5. Experimento 2 — HTTPChaos: error rate del 10 % en data-service

El segundo experimento inyecta fallas de aplicación en lugar de latencia de red: un `HTTPChaos` con `target: Response` y `abort: true` sobre `data-service`. La diferencia conceptual con el laboratorio Docker original es importante: allí el error rate se simulaba con `if random.random() < 0.1: raise HTTPException(500)` dentro del código; aquí el código de la aplicación no tiene ningún `random()` y la falla es completamente externa a ella. Esa diferencia es justamente la que produce el primer hallazgo.

5.1 Correcciones al manifiesto del enunciado

El YAML original del fork no era ejecutable tal cual y se corrigió en tres puntos:

- **Cuatro objetos de chaos en un solo archivo** (`HTTPChaos` + `PodChaos` + `StressChaos` + `Schedule`): un `kubectl apply` los habría inyectado todos a la vez, haciendo imposible atribuir el efecto observado a una causa. Se separó en `experiment-2-http-error.yaml` (solo el `HTTPChaos`) y `experiment-2-alternativas.yaml` (los otros tres, documentados pero no aplicados). Es el mismo patrón de riesgo que ya se había corregido en el Experimento 1.
- **value: "100" con mode: fixed-percent**, es decir 100 % de error, no el 10 % que anunciaban los comentarios del propio archivo.
- **La granularidad del 10 % es imposible con 2 réplicas.** `HTTPChaos` no tiene un campo de probabilidad por request: el porcentaje se aplica a la selección de pods, y el pod seleccionado aborta el 100 % de sus propias requests. Con 2 réplicas el mínimo alcanzable es 50 %. Se escaló `data-service` a 10 réplicas y se usó `value: "10"` → 1 pod seleccionado → ~10 % de error agregado. Además `data-service-svc` pasó de `ClusterIP` a `NodePort`, porque `kubectl port-forward` fija la conexión a un solo pod y habría sesgado la medición hacia 0 % o 100 %.

5.2 Incidente de capacidad al escalar (y su corrección)

INCIDENTE DURANTE LA PREPARACIÓN

Al escalar a 10 réplicas por primera vez, 5 pods se programaron en `kube-cp`, que ya corría control-plane, Postgres, registry, Prometheus, Grafana y Jaeger sobre 3,8 GB de RAM. El nodo se saturó: load average por encima de 100, apiserver OOM-killed y en crash-loop, y `kubectl` respondiendo con `TLS handshake timeout`. El diagnóstico hubo que hacerlo por `ssh` a los nodos con `crictl ps -a`, precisamente porque la API de Kubernetes estaba caída.

Corrección: RAM de las 3 VMs aumentada (control-plane 4→8 GB, workers 2→4 GB) y, sobre todo, `nodeAffinity` con `node-role.kubernetes.io/control-plane` `DoesNotExist` en el Deployment de `data-service`. En la corrida definitiva las 10 réplicas quedaron 5 en `kube-w1` y 5 en `kube-w2`, ninguna en el control-plane.

Lección: el blast radius de un experimento no es solo el que declara el manifiesto de chaos. Escalar el objetivo es parte del experimento, y sin una restricción de scheduling el daño alcanzó al plano de control — el componente que uno necesita justamente para observar y revertir el experimento.

5.3 Diseño de la corrida y resultados

La corrida definitiva se automatizó en `scripts/run-exp2.sh` para que sea reproducible: preflight (verifica que no queden objetos de chaos vivos de corridas anteriores), 90 s de línea base, escalado, inyección, y 540 s de carga con 4 workers en paralelo — 300 s de chaos más 240 s de observación posterior al rollback

— con muestreo cada 15 s de RESTARTS, estado del HTTPChaos y contadores de Prometheus, recolección de trazas de Jaeger y limpieza automática. Total: 1 158 requests contra las 105 de la corrida exploratoria previa, que es lo que permite distinguir el 10 % nominal del ruido de muestreo.

FASE	N	ERRORES	P50 OK	P95 OK
Línea base (sin chaos)	232	0 — 0,0 %	17 ms	33 ms
Chaos activo (0-300 s)	503	47 — 9,3 %	16 ms	25 ms
Post-rollback (300-540 s)	423	27 — 6,4 %	15 ms	21 ms

El 9,3 % medido contra el 10 % nominal valida el diseño de la selección por pods. Pero la fila inferior es la que no debería existir: 240 s después del `duration` configurado el sistema seguía fallando.

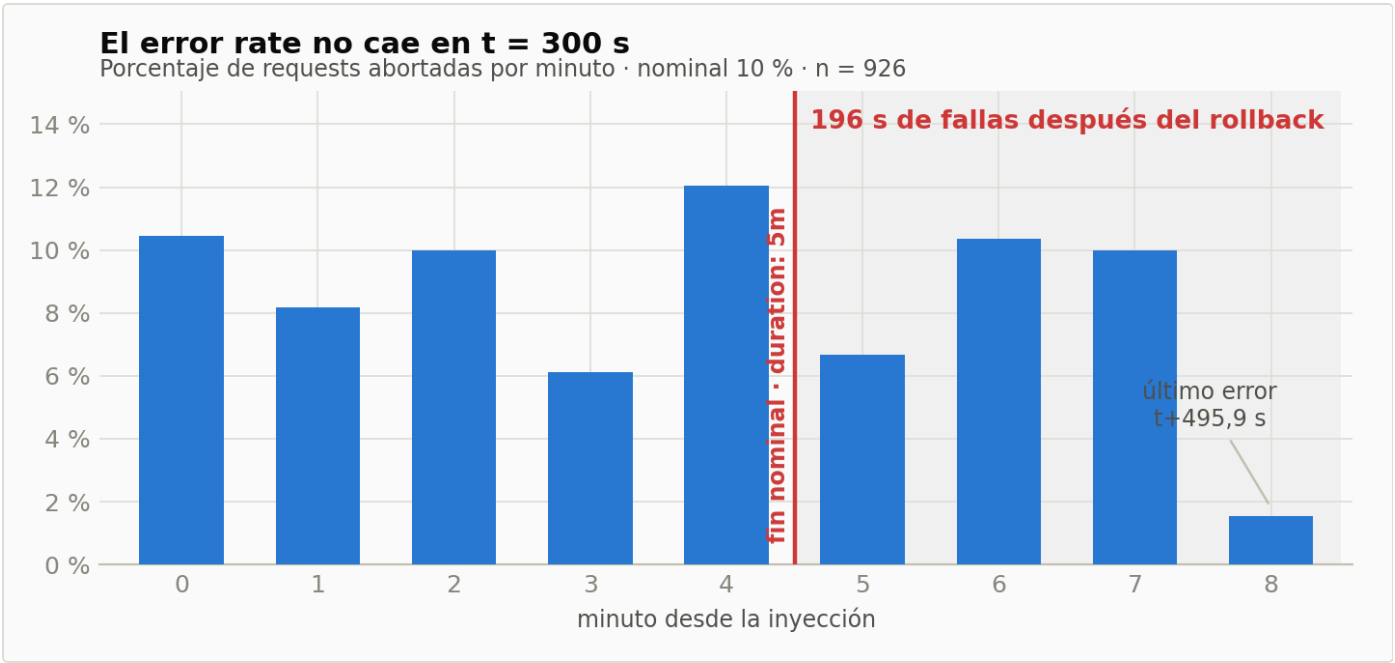


Figura 4. Error rate por minuto. El experimento declaraba `duration: 5m`; el último error se registró en t+495,9 s.

5.4 Hallazgo 1 — el tablero de trazas es ciego a la falla

Durante la ventana del experimento, Jaeger registró **1 113 trazas de data-service**, todas con `http.status_code = 200` (1 977 spans, cero 4xx y cero 5xx), mientras el cliente medía 74 requests fallidas en esa misma ventana.

El tablero de trazas es ciego a la falla inyectada

Error rate en la misma ventana de 568 s, según cada fuente de telemetría

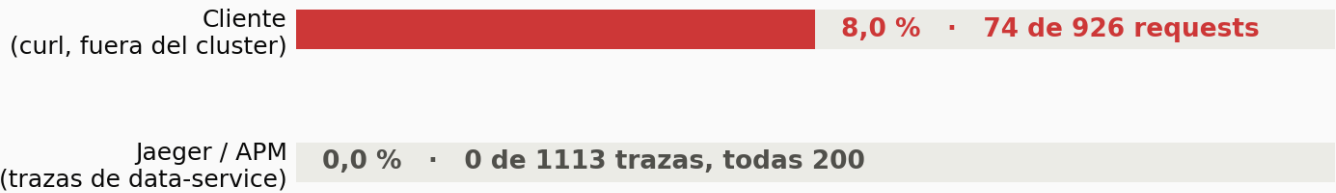


Figura 5. El mismo intervalo de 568 s medido por dos fuentes de telemetría distintas.

La explicación es estructural, no un fallo de configuración: HTTPChaos con `target: Response, abort: true` intercepta a nivel de proxy (tproxy + iptables en el network namespace del pod) *antes* de que la request llegue al código de la aplicación. La instrumentación de FastAPI nunca se ejecuta para esas requests, así que no hay span que exportar. La falla no aparece como error en el APM: aparece como **ausencia** de datos, que es exactamente lo que un tablero no muestra.

CONSECUENCIA PARA EL DISEÑO DE LA OBSERVABILIDAD

Un SLI de disponibilidad construido sobre trazas o sobre métricas emitidas por la propia aplicación reporta 100 % de éxito mientras el 10 % de los usuarios recibe conexiones abortadas. Detectar esta clase de falla exige telemetría *fuera* del proceso instrumentado: métricas del cliente o del borde (ingress, service mesh, blackbox exporter). El cliente además ve `http_code = 000` — un corte de conexión, no un código HTTP —, de modo que un SLI definido como «proporción de respuestas 5xx» tampoco lo cuenta.

Como detalle adicional, las requests abortadas se reparten en dos poblaciones bien separadas: 12 cortes prácticamente instantáneos (~0,02 s, RST inmediato) y 62 que se cuelgan **11,7 s** hasta el timeout de retransmisión TCP. Ninguna de las dos se parece a un error de aplicación, y la latencia de las requests exitosas no se degrada en absoluto — la falla es binaria, así que un SLI de latencia tampoco la habría detectado.

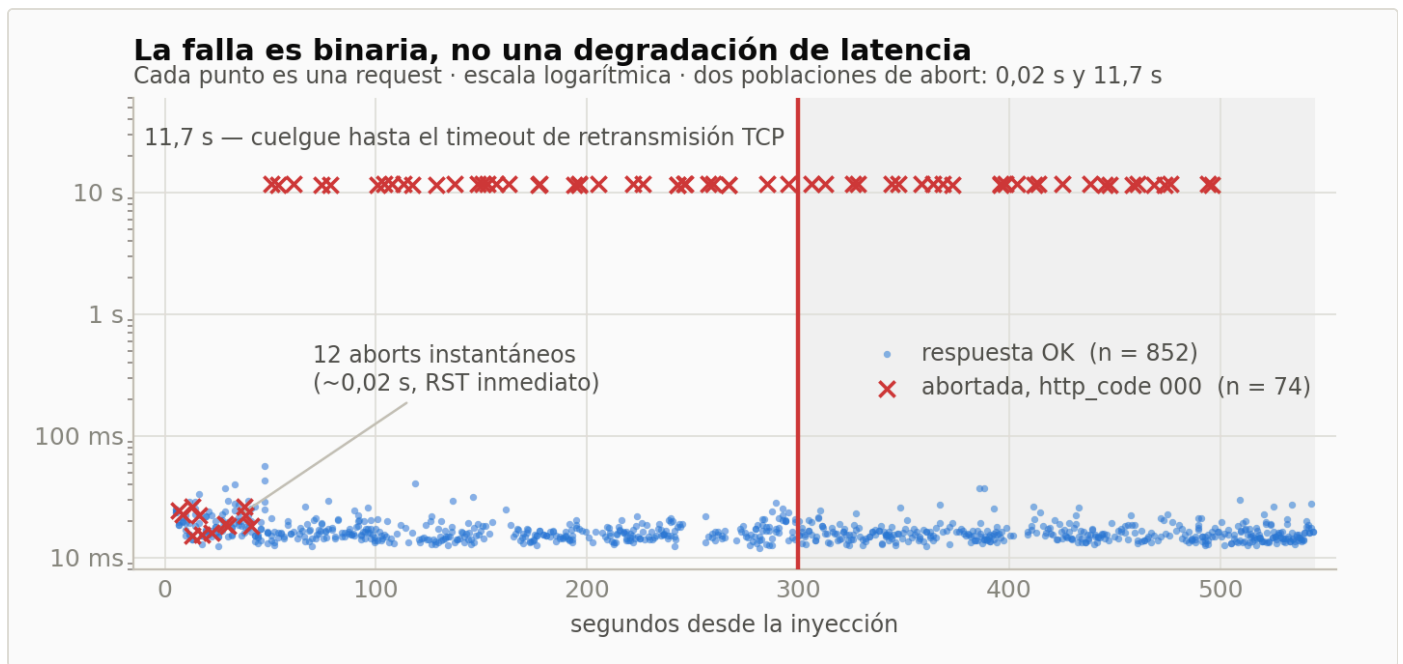


Figura 6. Latencia de cada request en escala logarítmica. Las exitosas se mantienen planas entre 15 y 25 ms durante todo el experimento.

5.5 Hallazgo 2 — el rollback no ocurrió, y por qué

A diferencia del Experimento 1, donde el TTL revirtió la falla de forma limpia, aquí el `duration: 5m` no detuvo nada: el último error se midió en **t+495,9 s**, esto es 196 s más allá del fin nominal, y el error rate se mantuvo en torno al 10 % por minuto hasta el minuto 7 del experimento. La causa se reconstruyó cruzando el muestreo de pods con los eventos del CRD.



Figura 7. Cadena causal del crash-loop y del rollback fallido.

El pod objetivo acumuló **6 reinicios** durante el experimento; los otros nueve pods de `data-service`, ninguno. El intervalo entre reinicios es de 80 s constantes, que es exactamente lo que predice la configuración de la sonda: `livenessProbe: httpGet /health` con `periodSeconds: 20` y `failureThreshold: 3` (60 s) más la terminación del contenedor. El manifiesto de chaos declara `path: "*" ,` de modo que el health check se aborta igual que el tráfico de negocio y el kubelet concluye, correctamente desde su punto de vista, que el contenedor está muerto.

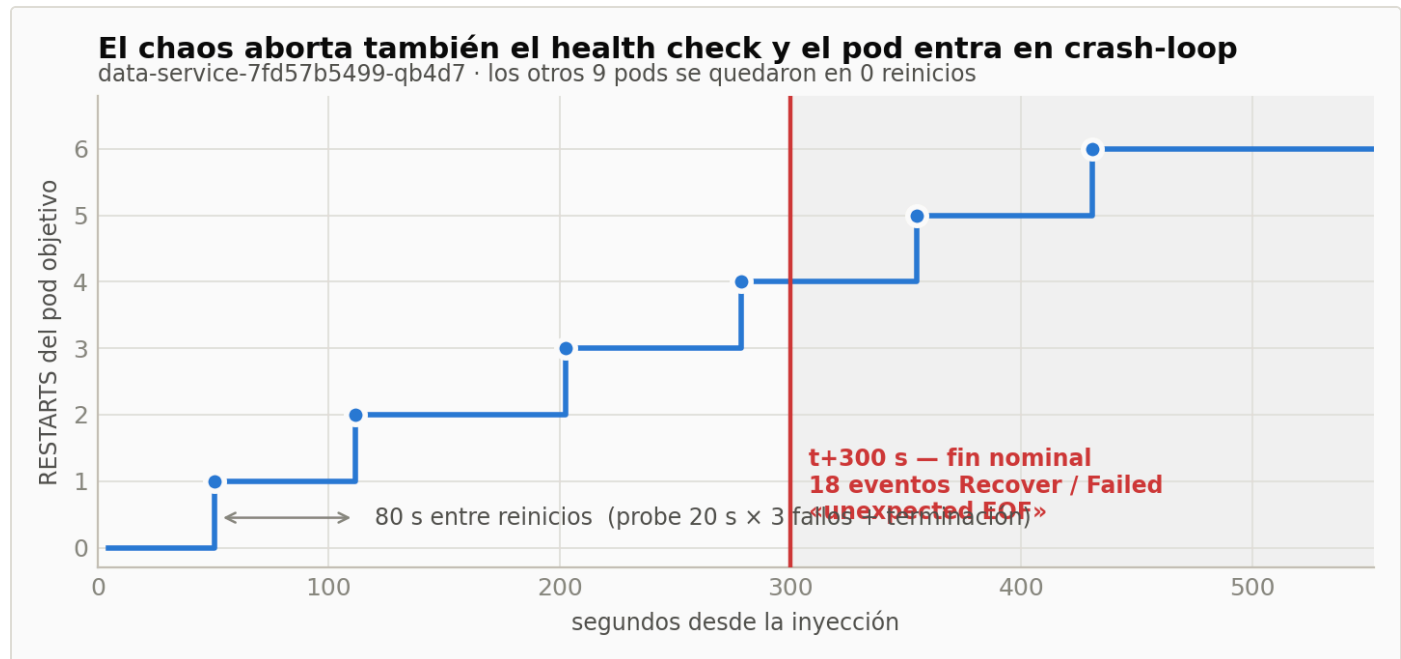


Figura 8. Reinicios acumulados del pod objetivo. Dos de los seis ocurrieron después del fin nominal del experimento.

Cada reinicio destruye el network namespace del contenedor y con él las reglas de iptables que Chaos Mesh había inyectado. Cuando a los 300 s el controlador intenta revertir, el chaos-daemon ya no encuentra el tproxy al que dirigirse: el `describe` registra **18 eventos Recover / Failed** con `apply config: send http request: unexpected EOF`, todos a partir del instante exacto del fin nominal. El record del experimento quedó en `phase: Injected/Wait` con `AllRecovered: False`.

EFFECTO SECUNDARIO: EL CRD NO SE DEJA BORRAR

El `finalizer` de Chaos Mesh espera a que el recovery termine antes de permitir el borrado del objeto. Como el recovery nunca puede completar, `kubectl delete httpchaos` queda colgado indefinidamente y el objeto sobrevive con `deletionTimestamp` pendiente. Fue necesario forzarlo:

```
kubectl -n otel-lab patch httpchaos error-rate-data-service-10pct --type=merge -p '{"metadata":{"finalizers":[]}}'
```

Quitar el finalizer borra el objeto de Kubernetes pero **no** revierte las reglas inyectadas: lo que realmente las elimina es destruir el pod, porque se van con su network namespace. La secuencia correcta de limpieza manual es quitar el finalizer, borrar el pod objetivo y recién entonces reescalar el Deployment.

LECCIÓN PRINCIPAL DEL GAME DAY

Un experimento de chaos cuyo blast radius incluye el health check del objetivo deja de ser una falla acotada y con duración: se convierte en un crash-loop con rollback no garantizado. El mecanismo de recuperación de la herramienta —el que hace que un Game Day sea seguro de correr en producción— depende de que el contenedor objetivo siga vivo, supuesto que el propio experimento invalida.

5.6 Remediaciones derivadas

1. **Excluir el health check del blast radius:** `path: "/data/*"` en vez de `path: "**"`, o una `livenessProbe` de tipo `exec / tcpSocket` que no atravesase el tproxy. Sin esto, ningún HTTPChaos sobre este servicio tiene rollback confiable.
2. **SLI de disponibilidad medido fuera del proceso:** blackbox exporter o métricas de ingress, en lugar de derivarlo de las trazas o de los contadores de la aplicación. El SLI actual habría reportado 100 % durante todo el incidente.
3. **Contar cortes de conexión como fallas,** no solo respuestas 5xx: el 100 % de los errores de este experimento llegaron al cliente como `http_code = 000`.
4. **Restricción de scheduling para todo objetivo de chaos** (`nodeAffinity` excluyendo el control-plane), de modo que escalar el objetivo no pueda tumbar el plano de control.
5. **Runbook de limpieza manual** con la secuencia finalizer → pod → réplicas, y un preflight que verifique que no quedaron objetos de chaos vivos antes de la siguiente corrida.
6. **Corregir las queries del tablero:** las métricas de la aplicación llegan a Prometheus a través del OTel Collector, de modo que el nombre real de la serie es `otelcol_data_requests_total` y no `data_requests_total`.

6. Conclusiones

La reproducción sobre infraestructura real confirma la hipótesis de estado estable definida ($p95 < 200$ ms en operación normal) y, en el Experimento 1, muestra degradación controlada y proporcional a la falla inyectada con recuperación automática al expirar el TTL sin intervención manual — los tres pilares de un Game Day (hipótesis, blast radius acotado, rollback automático) con evidencia real de Prometheus, Grafana y Jaeger.

El Experimento 2 es más interesante precisamente porque *no* se comportó como el diseño prometía, y los dos hallazgos que produjo solo aparecen al ejecutar sobre un cluster real:

- El error rate nominal se reprodujo con precisión (9,3 % medido contra 10 % esperado, $n = 503$), pero **la telemetría de la propia aplicación no vio absolutamente nada:** 1 113 trazas, todas 200. Una falla inyectada por debajo de la capa instrumentada es invisible para el APM, y esa es una propiedad del diseño de la observabilidad, no un error de configuración.
- **El rollback automático falló,** y la causa raíz —el chaos abortaba también el health check, provocando un crash-loop que destruía las reglas que Chaos Mesh necesitaba para revertirse— es un modo de fallo de la herramienta de chaos misma, no del sistema bajo prueba.

Ambos hallazgos apuntan en la misma dirección: **la garantía de seguridad de un Game Day («duración acotada, rollback automático») es condicional**, y las condiciones que la sostienen deben verificarse explícitamente antes de correr un experimento sobre un entorno que importe. Un experimento que no puede revertirse, en un sistema cuya telemetría no lo registra, es indistinguible de una caída real.

Evidencia: `results/exp2-20260829-225143/` — 1 158 requests, muestreo de pods y del CRD cada 15 s, eventos del `describe`, logs del chaos-daemon y trazas de Jaeger de la ventana. Corrida reproducible con `scripts/run-exp2.sh`.